

Demo Web Shop Capstone

Day-Wise Schedule and Testing Plan

Playwright JavaScript Automation | Wipro SDET Training

Ajay Bhatnagar

Application Under Test: Demo Web Shop

Final Coverage: 6 core modules and 2 add-on modules, 120 test cases, 360 passed executions

Field	Details
Website	https://demowebshop.tricentis.com/
Automation Tool	Playwright Test with JavaScript
Design Pattern	Page Object Model with service-wise spec files
Report Tool	Allure Report hosted on GitHub Pages
Final Hosted Report	https://ajaybhatnagar1712.github.io/Wipro_SDET_Playwright_Capstone/

Project Overview

This day-wise testing plan presents the capstone project work in a clean trainer-review format. The project validates Demo Web Shop using Playwright with JavaScript and follows a maintainable automation structure with page objects, service-wise tests, custom fixtures, Allure reporting, and GitHub Pages deployment.

The project satisfies the capstone requirement of 6 core modules and 2 add-on modules and minimum 120 test cases. Each service contains 15 unique test cases. The complete suite runs on Chromium, Firefox, and WebKit, producing 360 cross-browser executions in the final Allure report.

Capstone Requirement Mapping

Requirement	Implemented In Project
Minimum services	6 core modules and 2 add-on modules completed
Minimum test cases	120 unique Playwright test cases
Browser coverage	Chromium, Firefox, WebKit
Automation framework	Playwright Test with JavaScript
Project structure	POM structure: pages, tests, fixtures, data, api, docs
Report	Allure report generated and hosted using GitHub Pages
CI/CD	GitHub Actions workflow runs test suite and deploys report

Day-Wise Work Schedule

Day	Focus	Work Completed	Deliverable
Capstone Day 1	Project planning and website finalization	Selected Demo Web Shop, identified the 8 required services, created the test approach, and documented the scope for trainer review.	Testing plan, project objective, service list, and expected coverage.
Capstone Day 2	Playwright setup and base project structure	Created the Playwright JavaScript project with package files, configuration, report settings, and the standard folder layout.	playwright.config.js, package.json, tests folder, pages folder, fixtures folder.
Capstone Day 3	First two services implemented	Implemented Authentication and Product/Search services using visible spec files and page object	30 unique test cases and 90 cross-browser

		methods.	executions.
Capstone Day 4	Cart and Wishlist/Compare services	Added Cart and Wishlist/Compare coverage with 15 tests each and verified the module-level behavior locally before pushing.	60 unique test cases and 180 cross-browser executions.
Capstone Day 5	User Profile and Address/Shipping services	Added protected account page checks, register/login account validations, shipping information pages, and checkout progress labels.	90 unique test cases and 270 cross-browser executions.
Capstone Day 6	Checkout/Payment and Customer Support services	Completed remaining service coverage for checkout/payment pages, gift card fields, contact pages, sitemap, information pages, and footer links.	120 unique test cases and 360 cross-browser executions.
Capstone Day 7	Allure report and CI/CD hosting	Configured GitHub Actions to run the full Playwright suite, generate Allure output, and deploy the report through GitHub Pages.	Hosted Allure report with 360 passed executions.
Capstone Day 8	Documentation and final review	Updated README, project plan, day-wise schedule, and final capstone report. Verified project structure and final report links.	Final documentation set inside docs folder.

Detailed Day-Wise Plan

Capstone Day 1 - Project planning and website finalization

- Objective: Project planning and website finalization.
- Work completed: Selected Demo Web Shop, identified the 8 required services, created the test approach, and documented the scope for trainer review.
- Deliverable: Testing plan, project objective, service list, and expected coverage.

Capstone Day 2 - Playwright setup and base project structure

- Objective: Playwright setup and base project structure.
- Work completed: Created the Playwright JavaScript project with package files, configuration, report settings, and the standard folder layout.
- Deliverable: playwright.config.js, package.json, tests folder, pages folder, fixtures folder.

Capstone Day 3 - First two services implemented

- Objective: First two services implemented.
- Work completed: Implemented Authentication and Product/Search services using visible spec files and page object methods.
- Deliverable: 30 unique test cases and 90 cross-browser executions.
- Services covered: Authentication and Product/Search.

Capstone Day 4 - Cart and Wishlist/Compare services

- Objective: Cart and Wishlist/Compare services.
- Work completed: Added Cart and Wishlist/Compare coverage with 15 tests each and verified the module-level behavior locally before pushing.
- Deliverable: 60 unique test cases and 180 cross-browser executions.
- Services covered: Cart and Wishlist/Compare.

Capstone Day 5 - User Profile and Address/Shipping services

- Objective: User Profile and Address/Shipping services.
- Work completed: Added protected account page checks, register/login account validations, shipping information pages, and checkout progress labels.
- Deliverable: 90 unique test cases and 270 cross-browser executions.
- Services covered: User Profile/Account and Address/Shipping.

Capstone Day 6 - Checkout/Payment and Customer Support services

- Objective: Checkout/Payment and Customer Support services.
- Work completed: Completed remaining service coverage for checkout/payment pages, gift card fields, contact pages, sitemap, information pages, and footer links.
- Deliverable: 120 unique test cases and 360 cross-browser executions.
- Services covered: Checkout/Payment and Customer Support/Information.

Capstone Day 7 - Allure report and CI/CD hosting

- Objective: Allure report and CI/CD hosting.
- Work completed: Configured GitHub Actions to run the full Playwright suite, generate Allure output, and deploy the report through GitHub Pages.
- Deliverable: Hosted Allure report with 360 passed executions.

Capstone Day 8 - Documentation and final review

- Objective: Documentation and final review.
- Work completed: Updated README, project plan, day-wise schedule, and final capstone report. Verified project structure and final report links.
- Deliverable: Final documentation set inside docs folder.

Service-Wise Testing Plan

Service	Cases	Test Folder	Page Object
Authentication	15	tests/authentication	authentication.page.js
Product and Search	15	tests/product	product.page.js

Cart	15	tests/cart	cart.page.js
Wishlist and Compare	15	tests/wishlist-compare	wishlistCompare.page.js
User Profile and Account	15	tests/user-profile	userProfile.page.js
Address and Shipping	15	tests/address-shipping	addressShipping.page.js
Add on - Checkout and Payment	15	tests/add-on-checkout-payment	checkoutPayment.page.js
Add on - Customer Support and Information	15	tests/add-on-customer-support	customerSupport.page.js

Service Objectives

Authentication

Validate login, registration, password recovery, protected page redirects, and account links.

Product and Search

Validate home page, category pages, product details, search results, prices, and sorting controls.

Cart

Validate empty cart behavior, cart links, add-to-cart controls, order progress labels, and footer controls.

Wishlist and Compare

Validate wishlist, compare products, recently viewed products, new products, and email-a-friend pages.

User Profile and Account

Validate guest access protection, register page fields, login recovery, footer links, and sitemap account links.

Address and Shipping

Validate shipping/returns information, availability labels, shipping footer links, and checkout progress labels.

Add on - Checkout and Payment

Validate checkout guards, payment step labels, gift card fields, price information, and payment-related controls.

Add on - Customer Support and Information

Validate contact us, sitemap, privacy notice, conditions, about us, news, blog, newsletter, and footer links.

Automation Framework Structure

Path	Purpose
.github/workflows/allure-report.yml	CI/CD workflow for Playwright execution, Allure generation, and GitHub Pages deployment.
tests/	Contains 6 core module folders, 2 add-on module folders, and 120 visible spec files.
pages/	Contains page object classes used by the spec files.
fixtures/baseFixture.js	Creates reusable page object fixtures for service-level tests.
data/demoWebShopData.js	Stores base URL, service coverage, browser list, and expected result count.
playwright.config.js	Defines test directory, reporters, timeouts, retries, screenshots, traces, videos, and browsers.
docs/	Stores capstone plan, day-wise schedule, notes, and final report files.

Execution and Reporting Flow

1. Open the project folder in VS Code or terminal.
2. Install dependencies using npm install.
3. Run npx playwright test to execute the full suite.
4. Playwright runs each test on Chromium, Firefox, and WebKit.
5. Allure result files are generated in allure-results.
6. Allure HTML report is generated from the result files.
7. GitHub Actions deploys the report to the gh-pages branch.
8. GitHub Pages hosts the final Allure report for trainer review.

```
npm install
npx playwright test
npx allure generate allure-results --clean -o allure-report
npx allure open allure-report
```

Final Execution Result

Metric	Final Value
Unique test cases	120
Services covered	8

Browser projects	3
Cross-browser executions	360
Passed	360
Failed	0
Skipped	0
Hosted report	https://ajaybhatnagar1712.github.io/Wipro_SDET_Playwright_Capstone/

Browser	Executions	Passed
Chromium	120	120
Firefox	120	120
WebKit	120	120

Final Submission Checklist

- 6 core module folders and 2 add-on module folders are present under tests.
- Every service contains 15 Playwright spec files.
- Page object classes are present under pages.
- README explains project scope, structure, and report link.
- GitHub Actions workflow runs the full test suite.
- GitHub Pages is set to gh-pages branch and root folder.
- Allure report shows 360 passed executions.
- Documentation files are present inside docs folder.

Conclusion

The day-wise capstone plan shows a clear progression from project setup to full service coverage, CI/CD deployment, and final reporting. The project now meets the trainer requirement with 6 core modules and 2 add-on modules, 120 Playwright JavaScript test cases, and a hosted Allure report showing 360 passed cross-browser executions.

Detailed Testing Plan

Demo Web Shop Capstone Testing Plan

Playwright JavaScript automation using trainer-style POM project structure

Project Overview

This capstone validates Demo Web Shop using Playwright with JavaScript. The project now follows a visible Page Object Model structure like a normal automation framework: pages, fixtures, data, api, docs, screenshots, and service-wise tests.

Field	Details
Website under test	Demo Web Shop
Website URL	https://demowebshop.tricentis.com/
Framework	Playwright Test with JavaScript
Design pattern	Page Object Model with custom fixtures
Requirement	6 core modules and 2 add-on modules, 15 test cases each, 120 test cases total
Browser execution	Chromium, Firefox, WebKit
Total report executions	360
Report hosting	Allure report hosted on GitHub Pages

Project Structure

Path	Purpose
.github/workflows/allure-report.yml	CI/CD workflow for Playwright, Allure, and GitHub Pages.
api/apiClient.js	Reusable API helper kept for trainer-style project structure.
data/demoWebShopData.js	Project data such as base URL, service coverage, and browser list.
docs/	Project plan, schedule, and Playwright notes.
fixtures/baseFixture.js	Custom Playwright fixture that provides all page objects to tests.
pages/base.page.js	Common navigation and assertion helper methods.
pages/*.page.js	Service-wise Page Object Model classes.
screenshots/	Folder reserved for report screenshots or evidence.
tests/<module>/*.spec.js	Visible service-wise test cases. There are 120 spec files.
playwright.config.js	Browser projects, base URL, reporters, timeouts, and retry settings.

Service-Wise Coverage

Service	Cases	Test Folder	Page Object
Authentication	15	tests/authentication	authentication.page.js
Product and Search	15	tests/product	product.page.js
Cart	15	tests/cart	cart.page.js
Wishlist and Compare	15	tests/wishlist-compare	wishlistCompare.page.js
User Profile and Account	15	tests/user-profile	userProfile.page.js

Address and Shipping	15	tests/address-shipping	addressShipping.page.js
Add on - Checkout and Payment	15	tests/add-on-checkout-payment	checkoutPayment.page.js
Add on - Customer Support and Information	15	tests/add-on-customer-support	customerSupport.page.js

Execution Flow

- A test file imports the custom fixture from fixtures/baseFixture.js.
- The fixture creates the required page object from the pages folder.
- The test calls one page-object method for the matching test case.
- The page-object method navigates to Demo Web Shop and performs assertions.
- Playwright runs the same visible 120 tests on Chromium, Firefox, and WebKit.

Current Verified Result

Metric	Result
Functional test cases	120
Cross-browser executions	360
Status	360 passed, 0 failed, 0 skipped
Hosted report	https://ajaybhatnagar1712.github.io/Wipro_SDET_Playwrite_Capstone/